

Data Analysis with Python 2024–2025

Parte 3: Matplotlib

Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Aprender a desenhar figuras usando Matplotlib.

Conteúdo

1 Matplotlib

Para compreender melhor os dados, é útil conseguir visualizá-los de várias maneiras. Matplotlib é a biblioteca de visualização de baixo nível mais comum para Python. Ela pode criar gráficos de linhas, gráficos de dispersão, gráficos de densidade, histogramas, mapas de calor e outros tipos de visualização.

Durante este curso, não entraremos profundamente nos detalhes do Matplotlib. Em vez disso, veremos alguns exemplos distribuídos ao longo do restante do material para mostrar seu uso.

1.1 Figura simples

Começaremos com um exemplo. A forma padrão de importar Matplotlib é usando a abreviação `plt`. Primeiro, vamos criar alguns dados para visualizar.

```
import numpy as np
import matplotlib.pyplot as plt

a = np.array([2, 5, 7, 4, 7, 0, 3, 1, 9, 2])

plt.plot(a) # desenha os pontos do array a
plt.title("Minha primeira figura")
plt.xlabel("Meu eixo x")
plt.ylabel("Meu eixo y")
plt.show() # solicita ao matplotlib que mostre a figura
```

A função `plot` realiza o desenho do gráfico. As demais chamadas ajustam detalhes da figura. Em *notebooks*, `plt.show()` nem sempre é estritamente necessário, mas deixá-lo explícito pode tornar o código mais organizado.

É importante entender como os valores armazenados no array `a` correspondem às características apresentadas na figura.

1.2 Componentes de uma figura

Os principais componentes de qualquer figura do Matplotlib e sua terminologia podem ser visualizados na documentação da biblioteca. O objeto de nível superior é a **figure**. Uma figura pode conter uma ou mais subfiguras, que no Matplotlib são chamadas de **axes**.

No gráfico anterior, as coordenadas x foram definidas implicitamente pelos índices do array `a`, isto é, por `arange(10)`.

2 Subfiguras

É possível criar uma figura com várias subfiguras usando o comando `plt.subplots`. Ele cria uma grade de subfiguras; o número de linhas e colunas da grade é informado como parâmetro.

O comando retorna um par contendo:

1. o objeto **figure**;
2. um array contendo as subfiguras.

No Matplotlib, as subfiguras são chamadas de **axes**. Observe a diferença entre *axis*, no singular, e *axes*, no plural. Assim, é possível pensar em *axes* como o par formado pelos eixos x e y que define uma figura bidimensional.

Exemplo:

```
fig, ax = plt.subplots(2, 2)
print(ax.shape)

ax[0,0].plot(np.arange(6)) # superior esquerdo
ax[0,1].plot(np.arange(6, 0, -1)) # superior direito
ax[1,0].plot((-1)**np.arange(6)) # inferior esquerdo
ax[1,1].plot((-1)**np.arange(1, 7)) # inferior direito

plt.show()
```

Se você não quiser trabalhar diretamente com os objetos **axes**, pode direcionar as chamadas ao módulo **pyplot**, em um estilo mais parecido com **MATLAB**:

```
plt.subplot(2, 2, 1)
plt.plot(np.arange(6))
plt.subplot(2, 2, 2)
plt.plot(np.arange(6, 0, -1))
plt.subplot(2, 2, 3)
plt.plot((-1)**np.arange(6))
plt.subplot(2, 2, 4)
plt.plot((-1)**np.arange(1, 7))
plt.show()
```

Observe que, neste exemplo, usamos tanto a função **plt.plot** quanto o método **plot** de um objeto **axes**. As funções no espaço de nomes **plt** referem-se às variáveis globais que informam qual figura e quais eixos estão ativos.

Se precisarmos trabalhar com várias figuras e/ou vários eixos ao mesmo tempo, não podemos depender apenas das funções de **plt**. Nesse caso, devemos acessar diretamente cada objeto **figure** ou **axes** e utilizar seus métodos de desenho.

A ideia é semelhante ao uso de geradores de números aleatórios do NumPy. Funções como **np.random.randn** utilizam o gerador aleatório global. Quando queremos vários geradores independentes ao mesmo tempo, podemos criar os geradores e então usar seus métodos.

Assim, tanto para geradores aleatórios quanto para gráficos do Matplotlib, podemos escolher entre um único objeto global, com funções que se referem a ele, ou vários objetos independentes e seus respectivos métodos.

3 Outras bibliotecas de visualização de dados para Python

O desenvolvimento da biblioteca Matplotlib começou em 2003. Em alguns aspectos, essa idade pode ser percebida, pois as figuras padrão podem não parecer tão atraentes quanto as produzidas por alternativas mais modernas. Além disso, algumas figuras simples podem exigir uma quantidade considerável de código.

Algumas bibliotecas modernas e comuns são:

3.1 Seaborn

Seaborn é uma biblioteca de visualização de nível mais alto construída sobre o Matplotlib. Ela permite criar facilmente gráficos mais complexos. As figuras produzidas também tendem a ter uma aparência mais agradável do que aquelas obtidas com as configurações padrão do Matplotlib.

3.2 Bokeh

Bokeh cria páginas HTML que podem ser visualizadas em um navegador. Como seu resultado não se limita a imagens estáticas e pode conter JavaScript, os gráficos podem ser interativos.

Essa interatividade pode incluir zoom, deslocamento da visualização ou controles como botões e barras deslizantes que modificam a figura.

3.3 HoloViews

HoloViews é uma biblioteca de nível ainda mais alto construída sobre Bokeh e Matplotlib.

3.4 Plotly

Plotly é uma ferramenta poderosa para criação de gráficos interativos.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.